

Тулчейн

# Повторение: процесс сборки программы

- Перечислите основные этапы сборки С-программы
- Зачем нужен каждый из этапов?

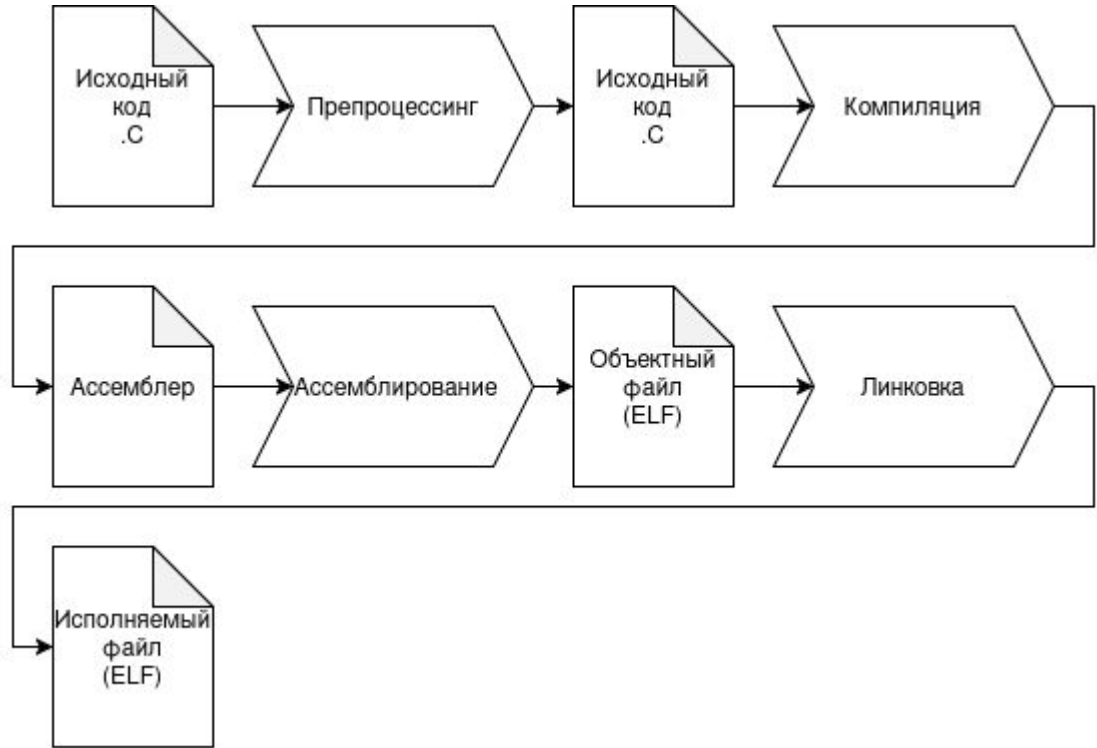
# Этапы сборки

**Преппроцессинг** – раскрытие директив препроцессора `#include`, `#define` и т.д.

**Компиляция** – трансляция исходного кода в ассемблерный код

**Ассемблирование** – трансляция ассемблерного кода в машинный байткод

**Линковка** – разрешение символов



# Преппроцессор

Преппроцессор работает на этапе до компиляции.

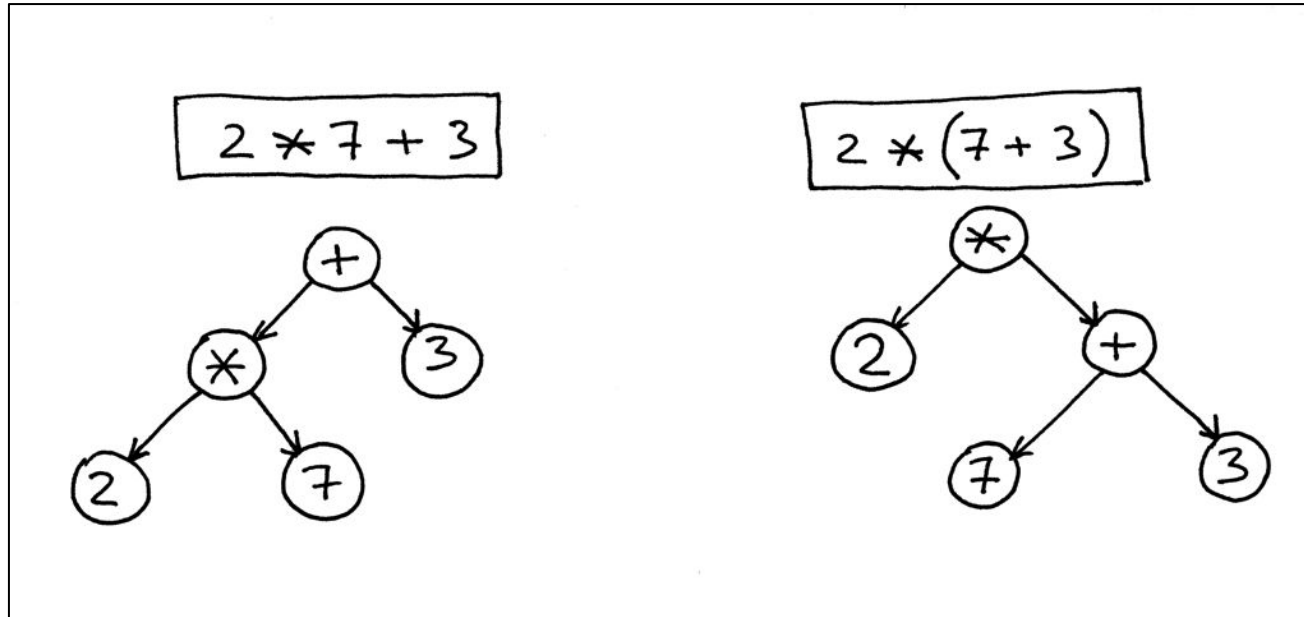
- Включает содержимое заголовочных файлов, чтоб мы каждый раз не писали прототипы функций заново
- Подставляет макросы
- Обеспечивает условную компиляцию

# Компиляция

Процесс трансляции исходного кода в ассемблерный. Компиляция тоже делится на этапы:

1. Лексический анализ, разбиение кода на токены (лексемы)
2. Семантический анализ + парсинг – построение дерева программы согласно грамматике
3. Рекурсивный обход дерева, трансляция каждого узла в ассемблер

# Компиляция. Пример простейшего AST



# Линковка

Часто программы состоят из нескольких файлов

Но компилятор за один раз обрабатывает только один файл

Как же нам вызвать функцию, объявленную в другом файле?

```
1 // a.c
2 int main() {
3     computation();
4 }
```

```
1 // b.c
2 void computation() {
3     printf("2 + 2 = %d\n", 2 + 2);
4 }
```

# Линковка

1. В **a.c** мы объявляем прототип функции **computation()** – сообщаем компилятору: “где-то в другом файле есть такая функция, а тут я хочу ее вызвать”
2. Файлы a.c и b.c компилируются **независимо**. В объектном файле a.o у нас есть неразрешенный вызов (т. е. вызов хз чего) для символа computation. Но символ никуда не указывает.
3. Линкер берет a.o и b.o и видит: в a.o вызывается символ, который есть в b.o. Значит, теперь нам известно что вызывать (**разрешает символ**). Потом он **объединяет секции** .text, .data и др. и получает единый исполняемый файл

```
1 // a.c
2 int main() {
3     computation();
4 }
```

```
1 // b.c
2 void computation() {
3     printf("2 + 2 = %d\n", 2 + 2);
4 }
```



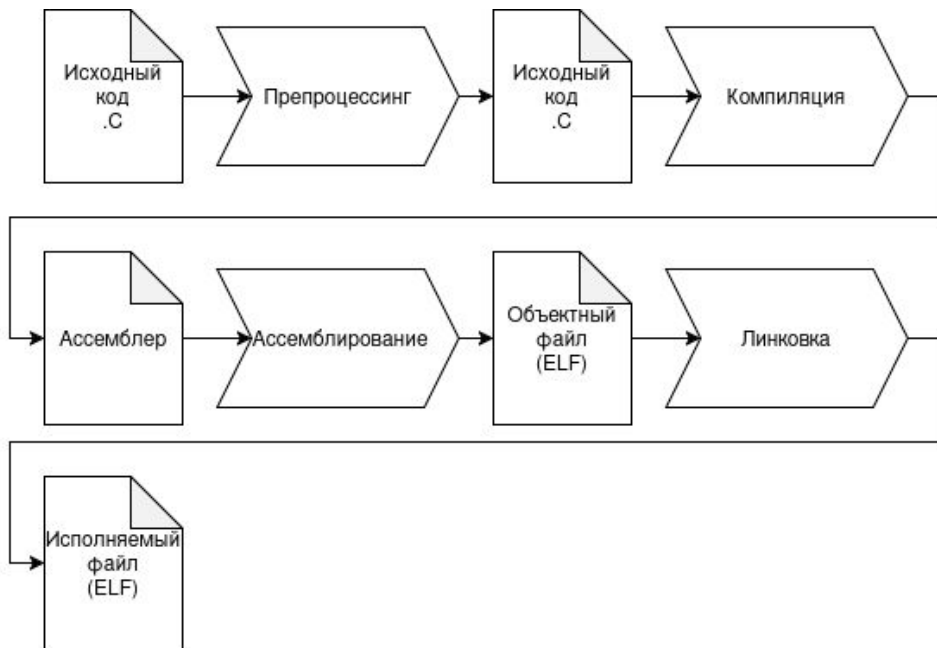
# Линковка

1. Что будет если символа с таким именем не найдется?
2. Что будет если найдется несколько символов с таким именем?

# Процесс компиляции

Процесс компиляции качественно замаскирован. Драйвер компиляции делает все это за нас

**Драйвер компиляции** – это обертка, которая упрощает нам жизнь. Она принимает файлы с кодом и флаги, автоматически вызывает все подкапотные программы и возвращает нам бинарный файл



# Процесс компиляции

Примеры драйверов компиляции:

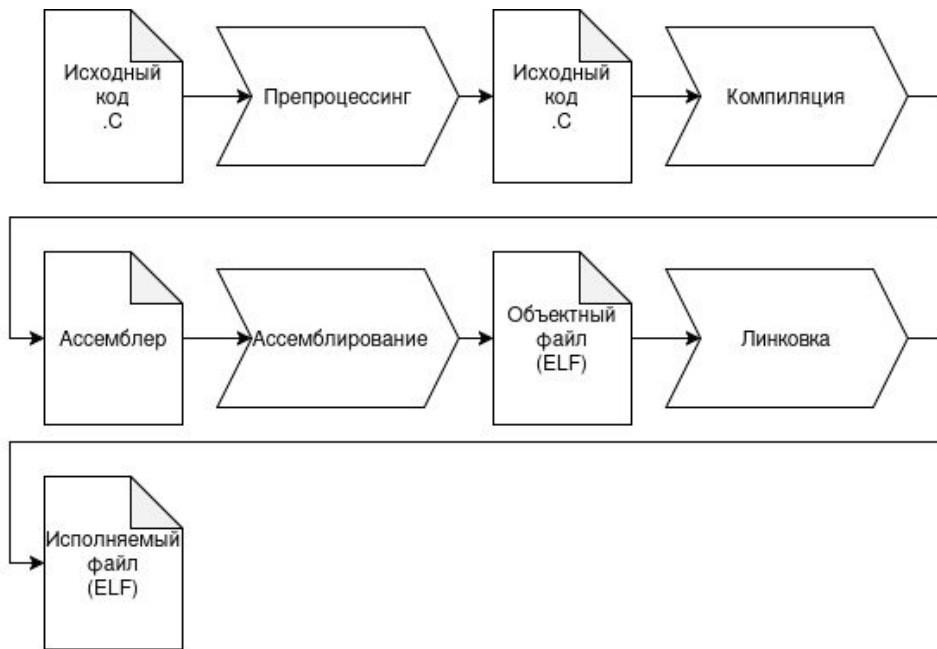
clang (llvm), gcc (gnu),  
mvcc (microsoft)

Рассмотрим на примере llvm:

1. драйвер – clang
2. компилятор – cc1
3. ассемблер – gas (gnu)
4. линкер – lld

И все это вызывается:

clang foo.c -o my-program



# Вызов компилятора

Драйвер вызывается так:

`clang <опции командной строки> <входные файлы>`

По умолчанию, компилятор генерирует исполняемый файл `a.out`, но это поведение меняется флагами. Основные флаги:

- o <имя выходного файла> – назвать выходной файл как вы хотите
- Wall – включить дополнительные предупреждения об ошибках
- O<число 0-3> – включить уровень оптимизаций от 0 (выключены) до 3 (небезопасные оптимизации)

И много других...

# Сборка программ

Представим себе маленькую программу на 3 файла: main.cpp, utils.cpp, core.cpp

Пусть мы поменяли один файл. Тогда нужно заново вызывать строку компиляции:

```
clang++ main.cpp utils.cpp core.cpp -o my-program
```

А что если у нас есть флаги?.. **Много** флагов для самых разных проверок

# Сборка программ

```
clang++ -D NDEBUG -ggdb3 -O0 -std=c++14 -Wall -Wextra -Weffc++ -Waggressive-loop-optimizations -Wc++0x-compat
-Wc++11-compat -Wc++14-compat -Wcast-align -Wcast-qual -Wchar-subscripts -Wconditionally-supported
-Wconversion -Wctor-dtor-privacy -Wempty-body -Wfloat-equal -Wformat-nonliteral -Wformat-security
-Wformat-signedness -Wformat=2 -Winline -Wlarger-than=32542 -Wlogical-op -Wnon-virtual-dtor -Wopenmp-simd
-Woverloaded-virtual -Wpacked -Wpointer-arith -Wredundant-decls -Wshadow -Wsign-conversion -Wsign-promo
-Wstack-usage=8192 -Wstrict-null-sentinel -Wstrict-overflow=2 -Wsuggest-attribute=noreturn
-Wsuggest-final-methods -Wsuggest-final-types -Wsuggest-override -Wswitch-default -Wswitch-enum -Wsync-nand
-Wundef -Wunreachable-code -Wunused -Wuseless-cast -Wvariadic-macros -Wno-literal-suffix
-Wno-missing-field-initializers -Wno-narrowing -Wno-old-style-cast -Wno-varargs -fcheck-new
-fsized-deallocation -fstack-protector -fstrict-overflow -fcto-odr-type-merging -fno-omit-frame-pointer -fPIE
-fsanitize=address -fsanitize=bool -fsanitize=bounds -fsanitize=enum -fsanitize=float-cast-overflow
-fsanitize=float-divide-by-zero -fsanitize=integer-divide-by-zero -fsanitize=leak -fsanitize=nonnull-attribute
-fsanitize=null -fsanitize=object-size -fsanitize=return -fsanitize=returns-nullable-attribute -fsanitize=shift
-fsanitize=signed-integer-overflow -fsanitize=unreachable -fsanitize=vla-bound -fsanitize=vptr main.c utils.c
core.c -o my-program
```

# Сборка программ

Не хочется каждый раз искать в консоли команду компиляции, либо набивать ее заново. Давайте поместим это в `bash`-скрипт: файл с командами консоли.

В чем может быть проблема такого решения?

# Сборка программ. Системы сборки

bash-скрипты это отличное решение для небольших проектов.

Как только в вашем проекте оказываются 1000 файлов, компиляция может занимать часы. Допустим, мы внесли изменения в `main.cpp`. Это значит, что нужно:

1. Перекомпилировать `main.cpp` – заново получить ассемблер
2. Перелинковать весь проект – взять новый `main.o` и старые `utils.o`, `core.o`.

Нам не нужно заново получать ассемблер из `utils.cpp`, `core.cpp`, мы можем воспользоваться уже полученными объектными файлами!



# Сборка программ. Система сборки make

Нам не нужно заново получать ассемблер из `utils.cpp`, `core.cpp`, мы можем воспользоваться уже полученными объектными файлами!

Именно этим и занимается система сборки Make. Она проверяет время последнего изменения исходного файла и не перезапускает компиляцию для этого файла, если он не изменился с момента последней компиляции

# Сборка программ. Система сборки make

Основной функционал make выглядит так:

в специальном файле Makefile мы описываем правила компиляции для проекта. Затем запускаем компиляцию. Синтаксис файла такой:

```
target: dependencies  
    rule
```

При исполнении цели target будут исполнены цели dependencies, а потом правило сборки rule

# Сборка программ. Система сборки make

Простейший Makefile:

```
all:
```

```
    g++ main.cpp utils.cpp core.cpp -o my-program
```

При выполнении `make all`, будет вызвана команда компиляции. Но сейчас это ничем не отличается от `bash`-скрипта. У цели `all` нет зависимостей, при изменении одного из файлов, всегда будут перекомпилироваться все

# Сборка программ. Система сборки make

```
1 all: my-program
2
3 my-program: main.o utils.o core.o
4 | g++ main.o utils.o core.o -o my-program
5
6
7 main.o: main.cpp
8 | g++ -c main.cpp
9
10 utils.o: utils.cpp
11 | g++ -c utils.cpp
12
13 core.o: core.cpp
14 | g++ -c core.cpp
15
```

# Сборка программ. Система сборки make

```
1 CC=g++
2 CFLAGS=-c -Wall
3 LDFLAGS=
4 SOURCES=utils.cpp main.cpp core.cpp
5 OBJECTS=$(SOURCES:.cpp=.o)
6 EXECUTABLE=my-program
7
8 all: $(SOURCES) $(EXECUTABLE)
9
10 $(EXECUTABLE): $(OBJECTS)
11 | $(CC) $(LDFLAGS) $(OBJECTS) -o $@
12
13 .cpp.o:
14 | $(CC) $(CFLAGS) $< -o $@
15
16
```

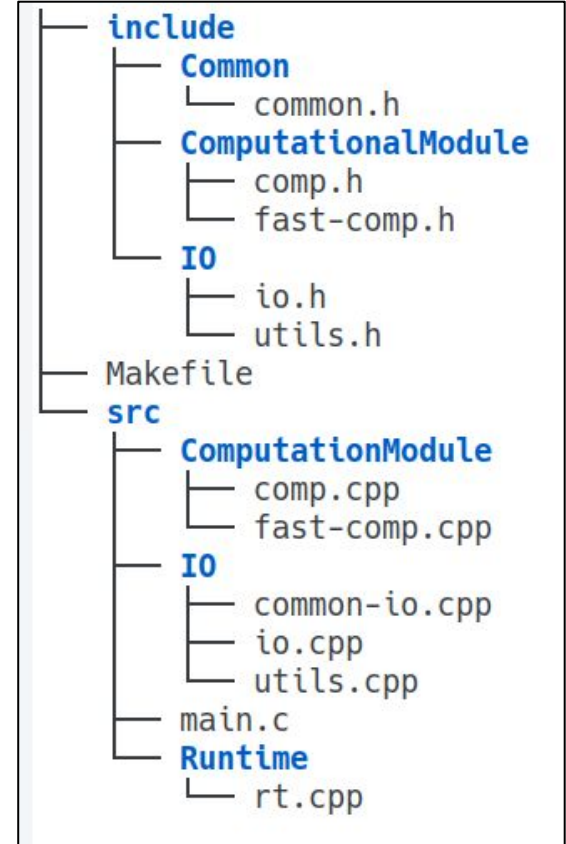
# Сборка программ. Организация проекта

Организация структуры проекта – важная часть работы. В проекте должно быть удобно ориентироваться и быстро находить нужные файлы.

Обычно структура такая:

Верхнеуровнево есть директория **src** с исходными файлами и **include** с заголовочными файлами

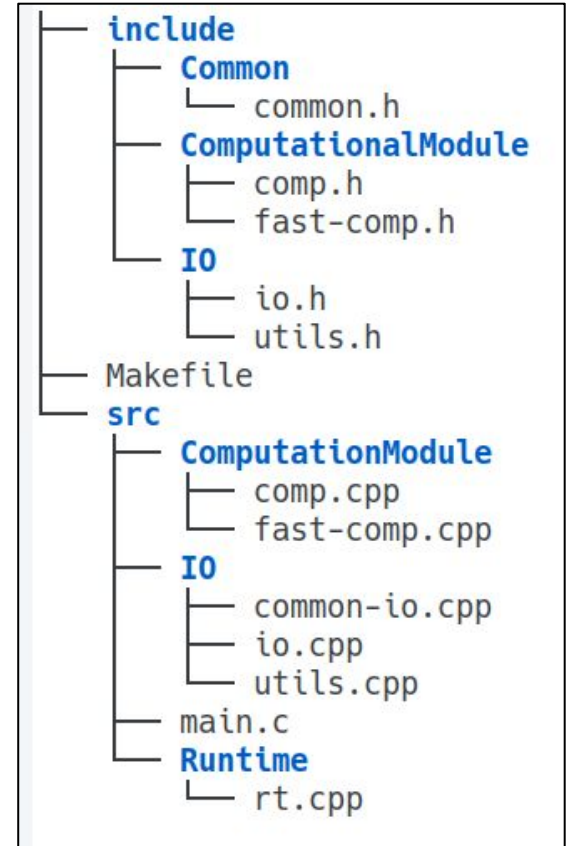
Эти директории подразделяются на логические модули. Обычно структура модулей одинаковая в **include/** и в **src/**



# Сборка программ. out-of-tree сборка

Считается не очень красивым, если ваша система сборки выдает объектики прямо рядом с исходным кодом: так их сложнее искать, можно что-то случайно удалить, директория проекта становится беспорядочной

Принято создавать папку build/, в которую и сохраняются все объектные и исполняемые файлы



# Сборка программ. Включение хедеров

Хедеры не просто так упакованы в отдельную директорию. Дело в том, что считается плохим тоном использовать относительные пути в инклюдях:

```
#include "../..../include/I0/io.h"
```

Программа становится чувствительна к структуре проекта, а это плохо.

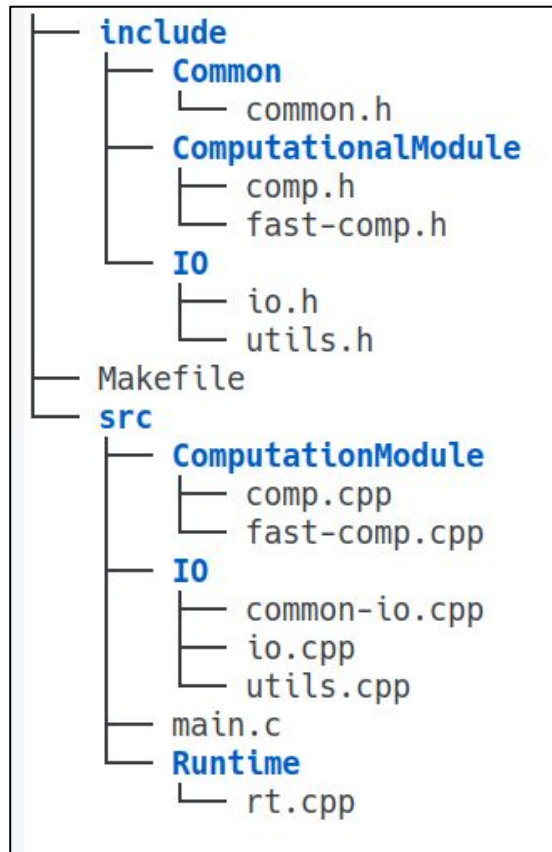
Чтоб избавиться от этого, есть флаг компиляции `-I<include-dir>`. Он говорит компилятору: "ищи хедеры в такой-то директории тоже".

Пример выше заменится на:

```
#include "I0/io.h"
```

И во время компиляции (в Makefile):

```
clang -I../..../include io.c -o io.o
```





# Аргументы командной строки

Мы заметили, что компилятор использует опции командной строки, чтоб получать входные данные и настройки работы. Можем ли мы делать также в наших программах?

Можем! `main` принимает два аргумента – количество опций командной строки и указатель на массив строк с этими аргументами. Мы можем обрабатывать их при помощи вызова `getopt` или вручную.

`argv[0]` – имя исполняемого нашего файла, всегда существует

```
4  
3 int main(int argc, const char *argv[]) {  
2 | SquareEqT equation;
```